

How many people can ask EdgeQuake at once?

We pointed a load generator at the shared demo and kept adding people until answers got too slow. Everything below is measured, on one named Google Cloud server, and explained in plain English.

RUN 2026-09-29-highload-push	TARGET https://demo.edgequake.com	PRODUCT VERSION 0.28.3
SERVER elitizon-db · e2-medium	TENANT 00000000-0000-0000-0000-000000000002	WORKSPACE 00000000-0000-0000-0000-000000000003

SHORT BURST (~2 MIN)

12 people

fresh questions, ≥90%
answered in 25 s (typical
17.1 s)

**SUSTAINED FOR
MINUTES**

8 people

after the ~2-minute burst
allowance; steady stream
up to 30/min

REPEATED FAQ

100%

answered from cache up
to 32 at once

**4-MINUTE NON-STOP
TEST**

Degraded

24% on time at the burst-
crowd size

The short answer. For a short burst the demo handled up to **12 people** asking fresh questions at once (it first missed at 14). Kept up for 3.5 minutes on end, it sustained a non-stop crowd of **8** and a steady stream of **30 questions a minute**. The VM was the bottleneck: its CPU sat at 90% or more for 32% of the measured time (peak 100%), and the “look up” step running on the VM grew from 1.2 s to 20.0 s per answer as the crowd grew, while the external LLM steps barely moved (7.3 s → 7.9 s).

Results at a glance

CARD	QUESTION	ANSWER
H1	How many people can ask fresh questions at the same moment (short test)?	up to 12 people ; first miss at 14
H2	How fast can fresh questions arrive on their own schedule (90 s per rate)?	at least 36/min
H3	What if everyone asks the same FAQ?	100% from cache, up to 32 at once
H4	Can the burst-level crowd (12) be held for 4 minutes?	No — 24% on time
H5	What can it sustain once the burst allowance is spent?	8 people · 30/min

“On time” always means: answered successfully within 25 seconds. A level passes when at least 90% of its answers are on time.

1. The question, in plain English

A demo can feel fast when one person uses it and slow when a whole team does. We wanted to know **where that change happens**: how many people can ask *brand-new* questions at the same moment and still get an answer in about **25 seconds**, most of the time (at least **9 answers in 10**).

The point where it stops meeting that promise is called the **knee**. Below the knee, adding people costs little. Past it, everyone waits longer and fewer good answers come out per second.

2. The server that was tested

The public demo at **demo.edgequake.com** is **not** a serverless service and not a large cluster. It is **one small Google Cloud virtual machine** running four containers. The exact facts below were read from Google Cloud and from the VM itself at the start of the run.

Where the test runs: client → one GCP VM → external LLM

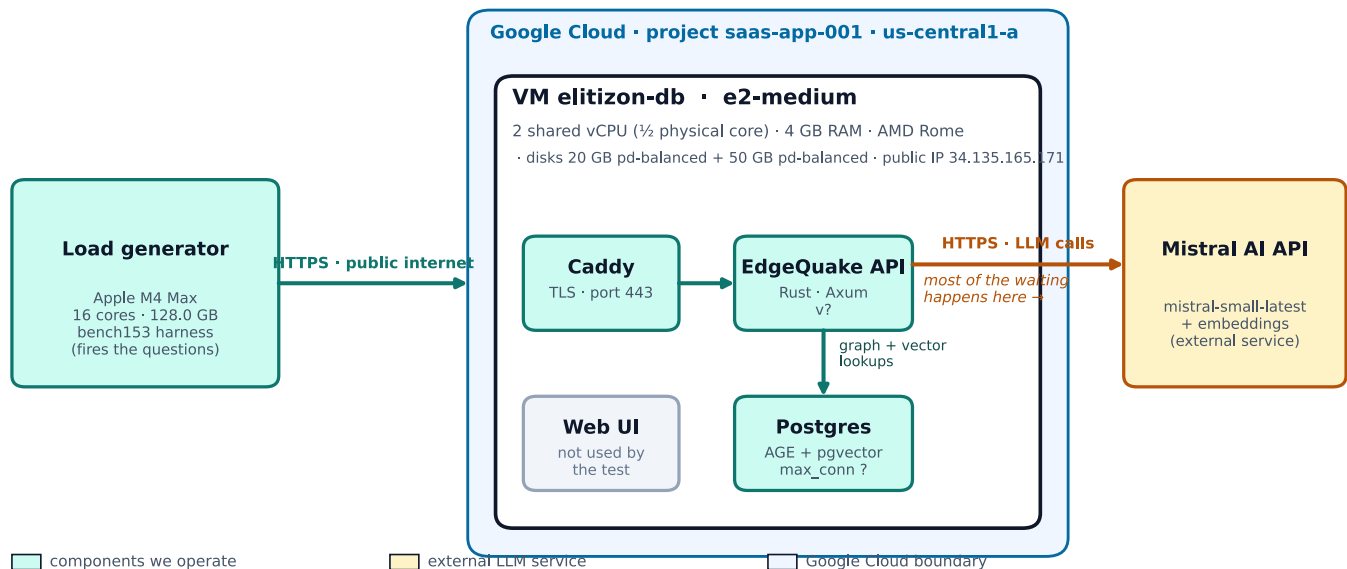


Figure 1 — The moving parts. Questions travel from the load generator over the internet to the VM. On the VM, Caddy passes them to the EdgeQuake API, which looks things up in Postgres and calls Mistral’s hosted LLM to understand the question and write the answer.

The virtual machine (system under test)

Provider / project	Google Cloud · project saas-app-001
Instance	elitizon-db (id 112371860905373219)
Zone	us-central1-a
Machine type	e2-medium — Efficient Instance, 2 vCPU (1/2 shared physical core) and 4 GB RAM
CPU	AMD Rome (AMD Rome) · 2 vCPUs, shared-core = True
Memory	4 GB (OS reports MB)
Disks	persistent-disk-0 — 20 GB pd-balanced (boot) elitizon-db-data — 50 GB pd-balanced (data)
Operating system	· kernel
Network	VPC edgequake-host-vpc / edgequake-host-subnet · internal 10.48.0.2 · public 34.135.165.171
DNS check	demo.edgequake.com → 34.135.165.171 (matches the VM’s public IP)
Scaling	Single VM, no autoscaling, no load balancer. Standard (non-preemptible), created 2026-09-18

Software and configuration on that VM

Containers on the VM	
EdgeQuake release	v0.28.3
Database	Postgres with Apache AGE (graph) + pgvector (embeddings) · max_connections , shared_buffers , work_mem
Answer model	mistral-small-latest via mistral (external API)
TLS / front door	Caddy reverse proxy terminates HTTPS; port 80 redirects to 443
Authentication	X-API-Key header (value never stored in this report)

Why this matters. An e2-medium gives two virtual CPUs but only one vCPU’s worth of CPU time in total (with short bursts above that), and 4 GB of memory that Postgres, the API, the web UI and Caddy all share. It is a cost-optimised demo box. Numbers here describe **this box and this LLM account**, not what EdgeQuake can do on production hardware. A Cloud Run service also named `edgequake-api` exists in the same project but is **not** what serves the demo.

The load generator (the machine that fired the questions)

Machine	Raphael-Mac-Book-Pro-Max.local
Hardware	Apple M4 Max · 16 logical cores · 128.0 GB
Software	macOS 27.0 · Python 3.12.14
Network path	the client’s own internet connection → public internet → Google Cloud us-central1

The load generator’s power is listed for honesty only. It sends small HTTPS requests, so it is not what limits the result.

3. How the test works — in plain English

Think of the demo as a coffee counter. One barista (the server) takes an order, phones a specialist supplier (the LLM), and hands the drink over. We want to know how big the queue can get before customers wait too long.

What happens when one person asks one question

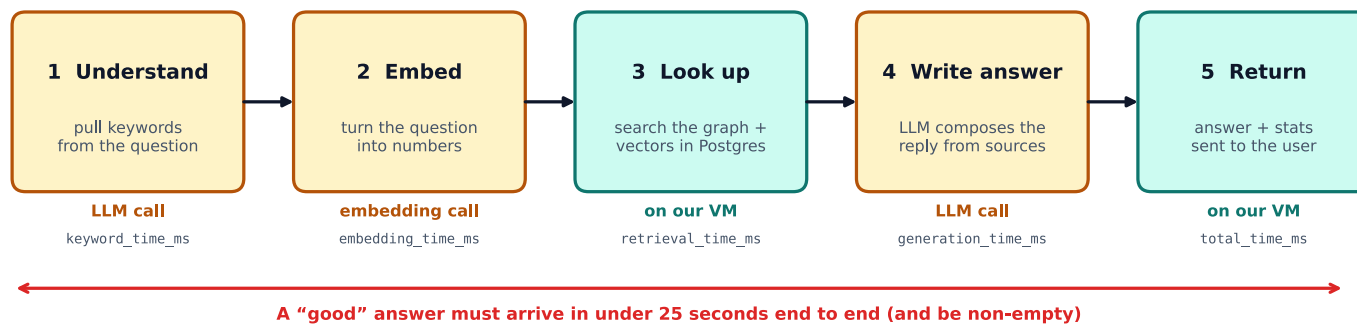


Figure 2 — One question, five steps. Amber steps call Mistral’s hosted LLM over the internet; teal steps run on our VM. The small grey labels are the exact timing fields the API returns, which we use to show where time goes.

The recipe

- 1 Check the counter is open.** We read `/health` and `/ready` and send one real, authenticated question. If the server is unhealthy or rejects our key, the run stops (BLOCKED) instead of measuring garbage.

- 2 **Use real, never-repeated questions.** Questions are about the papers already uploaded to the demo workspace. Every “fresh” question carries a one-off tag, so the server cannot reuse a stored answer. This measures real work, not memory.
- 3 **Turn the dial two different ways** (Figure 3): **H1** — a fixed crowd of 2, 4, 6, 8, 10, 12, 14, 16, 20 people each asking again as soon as they get an answer; **H2** — new questions arriving at a steady rate of 0.1, 0.15, 0.2, 0.25, 0.3, 0.35, 0.4, 0.5, 0.6 per second, whether or not earlier ones have finished.
- 4 **Score every answer.** An answer is **good** only if it came back successfully, was not empty, and took **under 25 seconds** (and, for fresh questions, was not a cache hit). A load level **passes** if at least 90% of its answers are good, fewer than 5% errors occur and fewer than 3 “too many requests” replies are seen.
- 5 **Find the knee.** The knee is the last level that passes before the first one that fails. Because you asked us to push further, we **kept going past the knee** to watch how the service degrades, with safety brakes: stop if over 25% of requests error, 10 or more “too many requests” replies appear, or under 10% of answers are good.
- 6 **Test the easy case separately (H3).** The same three FAQ questions are asked over and over by up to 32 people. Those are answered from a cache, so they are reported on their own and never mixed into the fresh-question capacity.
- 7 **Check it holds up (H4).** We hold the comfortable crowd steady for several minutes to see whether answers slowly get slower. This turned out to matter: a small shared-core VM is allowed to run flat-out only for about two minutes (see “burst vs sustained” below).
- 8 **Find the *sustained* ceiling (H5).** We rest the server for a few minutes so its burst allowance refills, apply one steady load for 3½ minutes, and judge only the part *after* the first two minutes. A load that still passes there is something the server can do all day.
- 9 **Watch the server while it happens.** Every few seconds we read the VM’s CPU, memory and per-container usage (read-only) to see whether the hardware or the LLM was the limit.

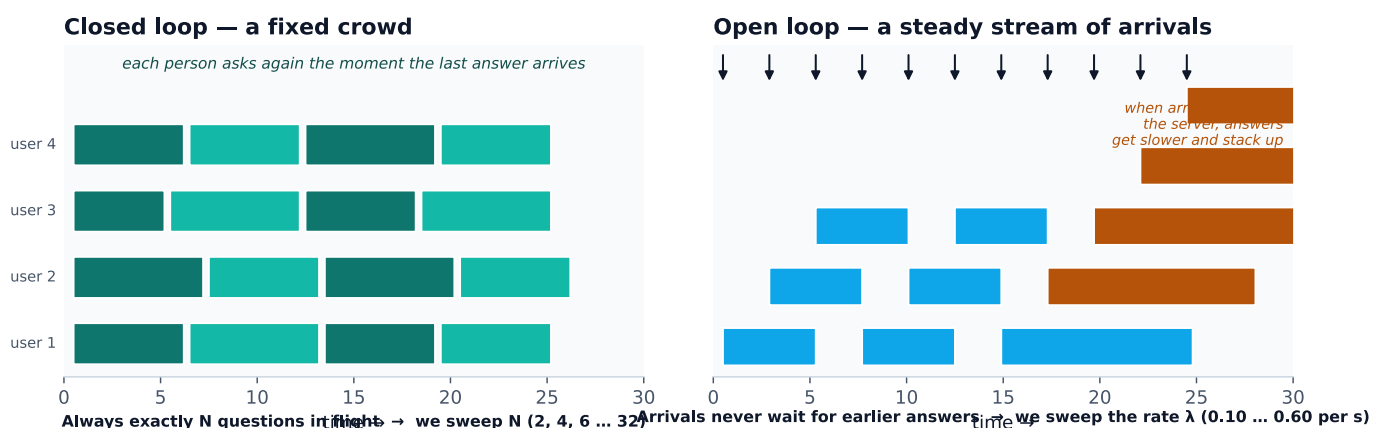


Figure 3 — Two ways to apply load. Left: a fixed crowd (closed loop) always keeps the same number of questions in flight. Right: a steady stream of arrivals (open loop) keeps coming even when the server is behind, so delays pile up. Real traffic looks more like the right-hand picture.

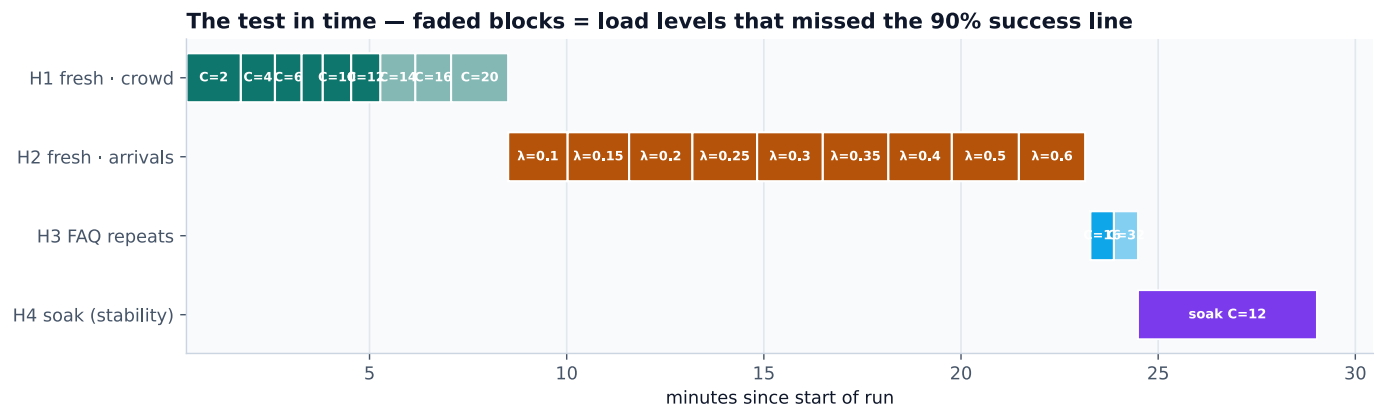


Figure 4 — What actually ran, and when. Each block is one load level. Faded blocks missed the 90% success line.

Rules of fair play

- One tenant and one workspace (IDs in the header), same model throughout.
- Fresh questions never repeat; FAQ tests are reported separately.
- Never more than 32 people at once, and never faster than 0.6 questions/s.
- Every request and its timings are saved (units.jsonl).

What we did not do

- No document uploads, deletes or OCR on the shared demo.
- We do not judge answer *quality* (that is a different benchmark).
- We do not claim this is “the” capacity of EdgeQuake — only of this VM and this LLM account, on this day.

4. Result 1 — a crowd of people asking fresh questions (H1)

In one sentence. Up to **12 people** asking at the same instant, at least 90% of fresh questions were answered within 25 seconds (typical wait 17.1 s, slowest-tenth starting at 22.9 s). The first level to miss was **14**. Good answers per second peaked at 0.52/s with 12 people at once. Beyond that, more people mostly added waiting rather than more answers. At the burst limit an answer spends about 8.0 s in the external-LLM steps and 6.8 s in the lookup on our own VM (Figure 7).

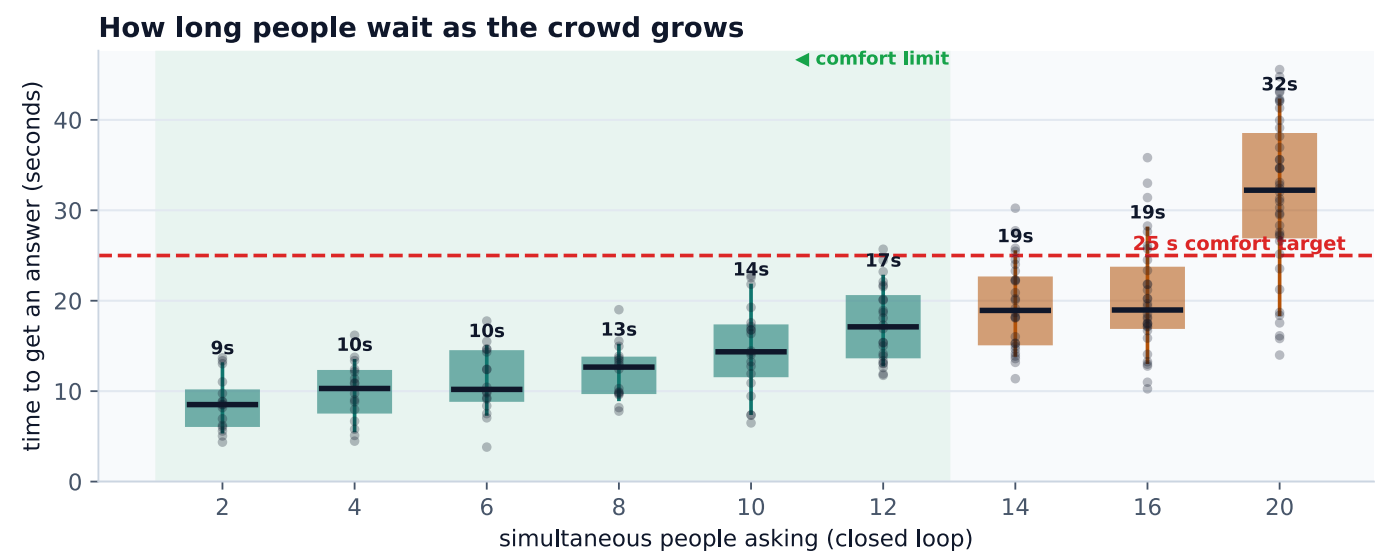


Figure 5 — Waiting time by crowd size. Each grey dot is one real answer. The bar spans the middle half of answers, the thin line spans the fastest to slowest tenth, and the thick tick is the typical (median) wait. Green shading marks the levels that stay within the promise.

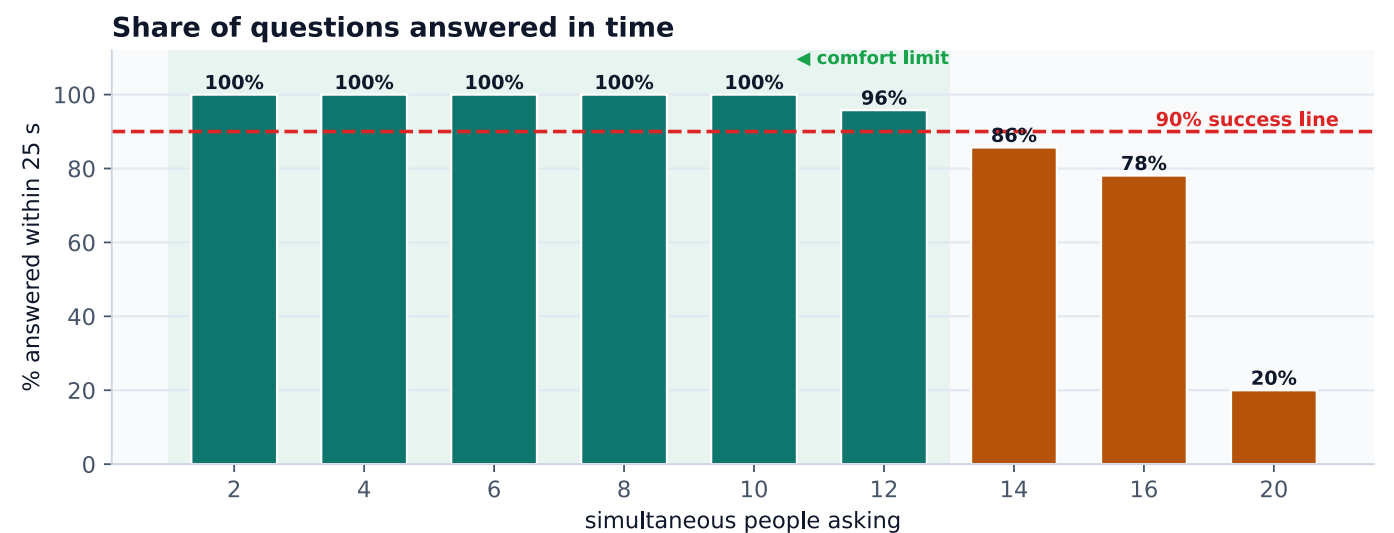


Figure 6 — Share answered in time. Teal bars pass the 90% line; amber bars miss it.

PEOPLE AT ONCE	QUESTIONS	ON TIME	TYPICAL WAIT	SLOWEST 10% FROM	GOOD ANSWERS/S	ERRORS	VERDICT
2	16	100%	8.5 s	13.2 s	0.19	0%	✓ comfortable
4	16	100%	10.3 s	13.6 s	0.31	0%	✓ comfortable
6	16	100%	10.2 s	15.1 s	0.39	0%	✓ comfortable
8	16	100%	12.7 s	15.2 s	0.50	0%	✓ comfortable
10	20	100%	14.4 s	21.9 s	0.46	0%	✓ comfortable
12	24	96%	17.1 s	22.9 s	0.52	0%	✓ comfortable
14	28	86%	18.9 s	25.6 s	0.45	0%	✗ over the limit
16	32	78%	19.0 s	28.2 s	0.46	0%	✗ over the limit
20	40	20%	32.2 s	42.4 s	0.09	0%	✗ over the limit

Where the time goes inside one answer — and how busy the VM is

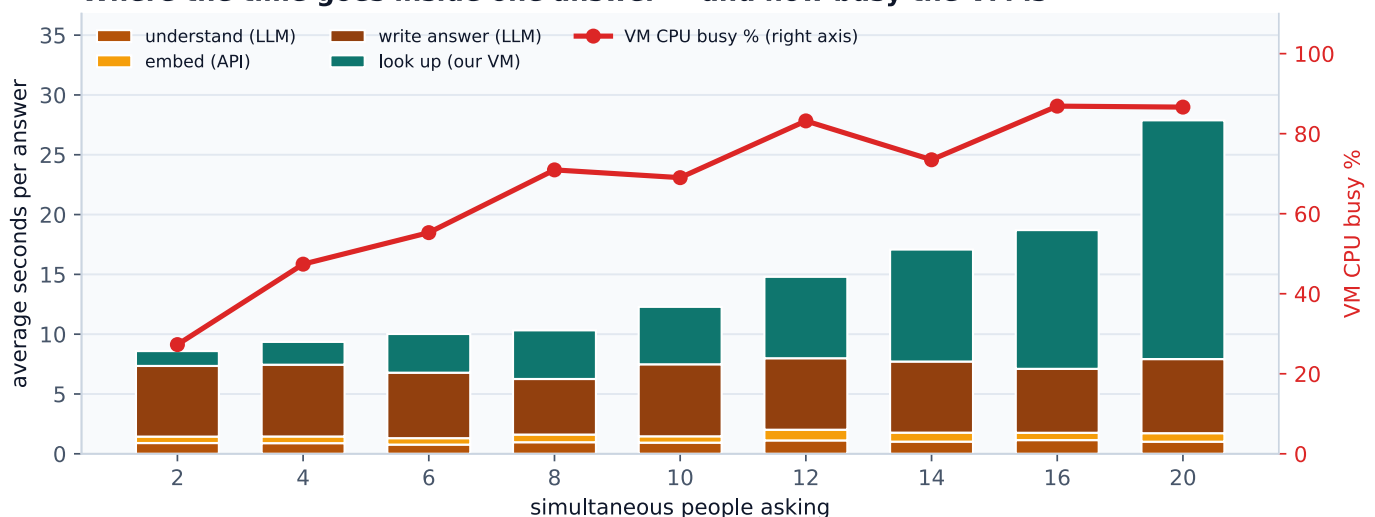


Figure 7 — Where the time goes. Amber/brown = calls to the external LLM and embedding service, teal = the lookup on our own VM, red line = how busy the VM's CPU was. If the amber part dominated, a faster VM would not help; if the teal part grows together with the red line, the VM is the limit.

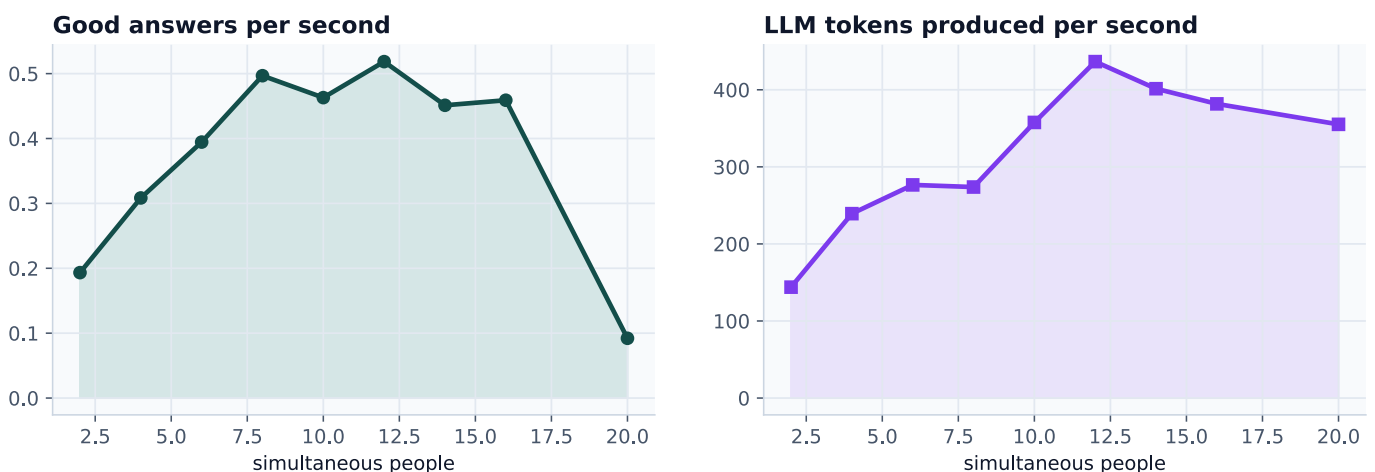


Figure 8 — Useful output. Left: good answers delivered per second. Right: tokens (word-pieces) generated per second by the LLM. When these flatten, the system is saturated no matter how many more people join.

5. Result 2 — questions arriving on their own schedule (H2)

In one sentence. For a 90-second window the demo kept up with a steady stream of about **36 fresh questions per minute** (0.6/s); even our top rate of 0.6/s (36/min) did not break it, so the true limit is higher than we tested for that short window (the sustained test below shows the longer-term limit).

In real life nobody waits politely for the person before them. Here new questions arrive at a fixed rate for 90 seconds per level. If the server is slower than the arrivals, the backlog grows and every later answer is later still.

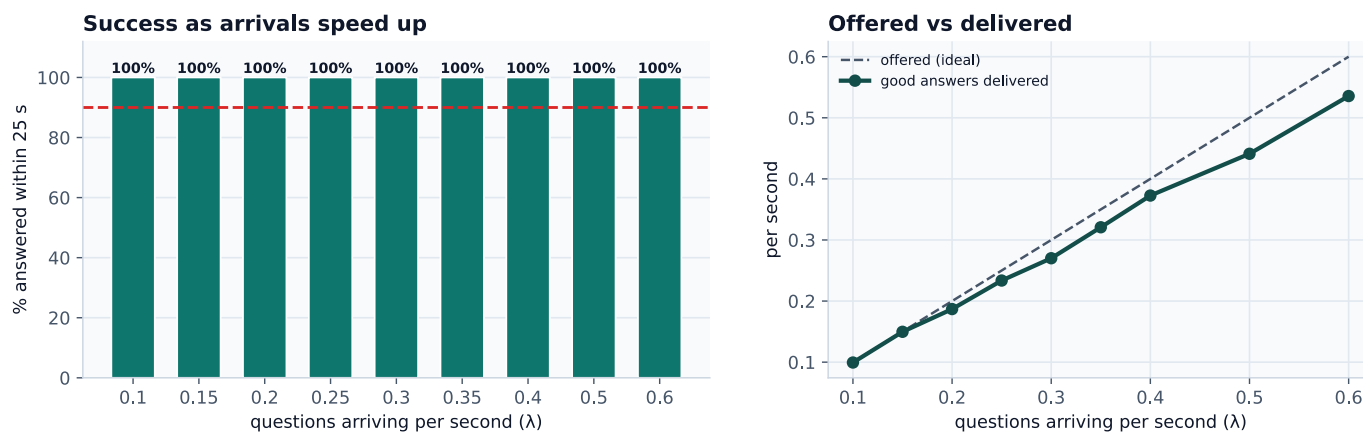


Figure 9 — Arrivals vs delivery. Left: share answered in time at each arrival rate. Right: the dashed line is what we offered; the teal line is what came back as good answers. When teal bends away from dashed, the server has hit its ceiling.

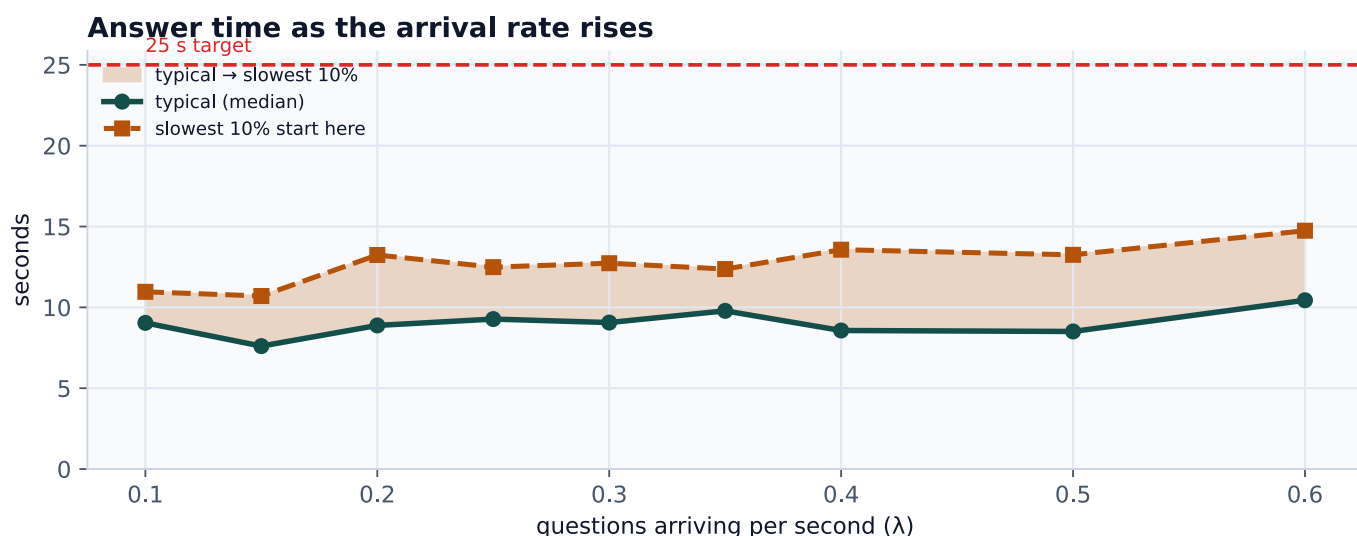


Figure 10 — The wait grows before it collapses. The shaded band runs from the typical answer to the slowest tenth.

ARRIVAL RATE	SENT	FINISHED	ON TIME	TYPICAL WAIT	SLOWEST 10% FROM	ERRORS	VERDICT
0.1/s (6/min)	9	9	100%	9.0 s	11.0 s	0%	✓ comfortable
0.15/s (9/min)	14	14	100%	7.6 s	10.7 s	0%	✓ comfortable
0.2/s (12/min)	18	18	100%	8.9 s	13.2 s	0%	✓ comfortable
0.25/s (15/min)	23	23	100%	9.3 s	12.5 s	0%	✓ comfortable
0.3/s (18/min)	27	27	100%	9.1 s	12.7 s	0%	✓ comfortable
0.35/s (21/min)	32	32	100%	9.8 s	12.4 s	0%	✓ comfortable
0.4/s (24/min)	36	36	100%	8.6 s	13.6 s	0%	✓ comfortable
0.5/s (30/min)	45	45	100%	8.5 s	13.2 s	0%	✓ comfortable
0.6/s (36/min)	54	54	100%	10.4 s	14.7 s	0%	✓ comfortable

What that means for people *online* (illustration, not a measurement)

People spend most of their time reading and thinking, not waiting. By Little's law, people online $\approx$ arrival rate $\times$ (thinking time + wait). With the sustained rate of 0.5 questions/s (30/min) and a typical 8.9 s wait:

BEHAVIOUR (ASSUMED)	TIME BETWEEN ASKS	PEOPLE ONLINE THE DEMO COULD CARRY
Very active (asks a question every ~30 s)	30s	$\approx$ 19 people
Typical (a question every ~2 min)	120s	$\approx$ 64 people
Casual (a question every ~5 min)	300s	$\approx$ 154 people

6. Result 3 — the same questions again and again (H3)

In one sentence. When 32 people asked the same three FAQs, **100%** were served from the answer cache with a typical wait of 18.5 s, versus 8.5 s for a fresh question on a quiet server.

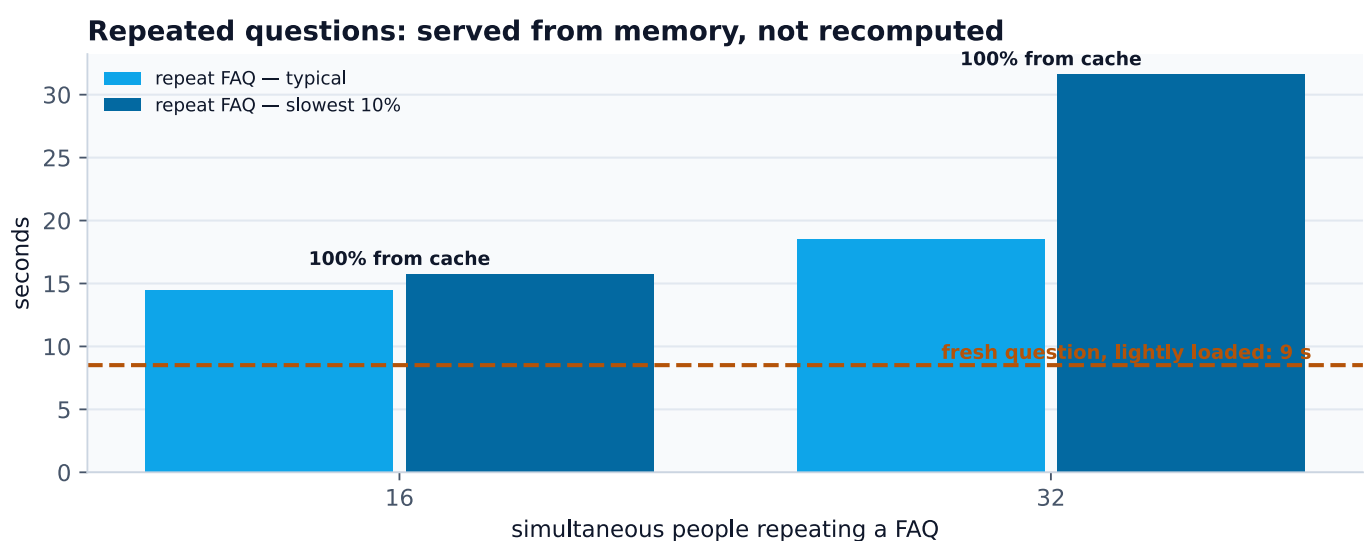


Figure 11 — Cache benefit. Repeated questions skip the LLM, so they come back quickly and cost nothing extra. The amber line is a fresh question when the server is quiet, for comparison.

Note: even cache hits show multi-second waits at high concurrency. That is queueing in front of the small VM, not LLM time, which is why the FAQ figures are reported separately and never counted as fresh-question capacity.

7. Result 4 — does it hold up over time? (H4 soak)

In one sentence. Holding **12 people** asking non-stop for 4 minutes produced 95 answers, 24% on time. Typical wait moved from 26.1 s (first third) to 35.0 s (last third), a drift of +34% — so the crowd size that looked fine for two minutes **could not be held**.

This is the most important surprise of the test. The short sweeps above are honest, but they are short. When the same crowd stays for several minutes, the server slows down — the explanation is in the next section.

Stability: 12 people asking non-stop for 4 minutes

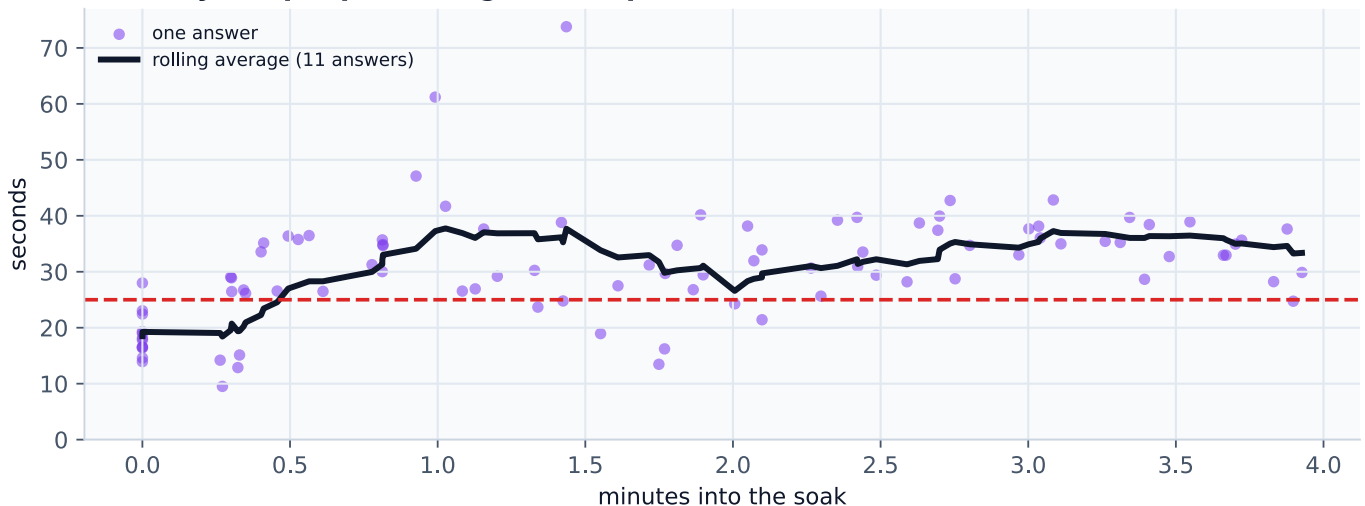


Figure 12 — Every answer during the soak. A flat rolling average means steady behaviour. A rising line means something (CPU allowance, memory, rate limits) was wearing out.

8. Result 5 — what the server can sustain (H5)

In one sentence. A non-stop crowd of **8 people** and a steady stream of **30 questions a minute** held up once the burst allowance was used up (3.5 minutes per level, judged after the first 2 minutes).

Burst vs sustained — the idea in plain English

The tested machine is a **shared-core** Google Cloud type (e2-medium). Google's documentation says these machines are guaranteed only **one vCPU's worth of CPU time in total**, and may borrow up to **two full vCPUs for roughly two minutes** when they have saved up credit while idle. Think of a phone plan with a fast-data allowance: everything is quick until the allowance runs out, then you are throttled.

So a load that looks fine in a two-minute sweep can be impossible to keep up. To measure the real, all-day capacity we (1) let the server **rest** for 150.0 s so the allowance refills, (2) apply one steady load for 210.0 s, and (3) **ignore the first 120 s** — the burst — and judge only what happens afterwards.

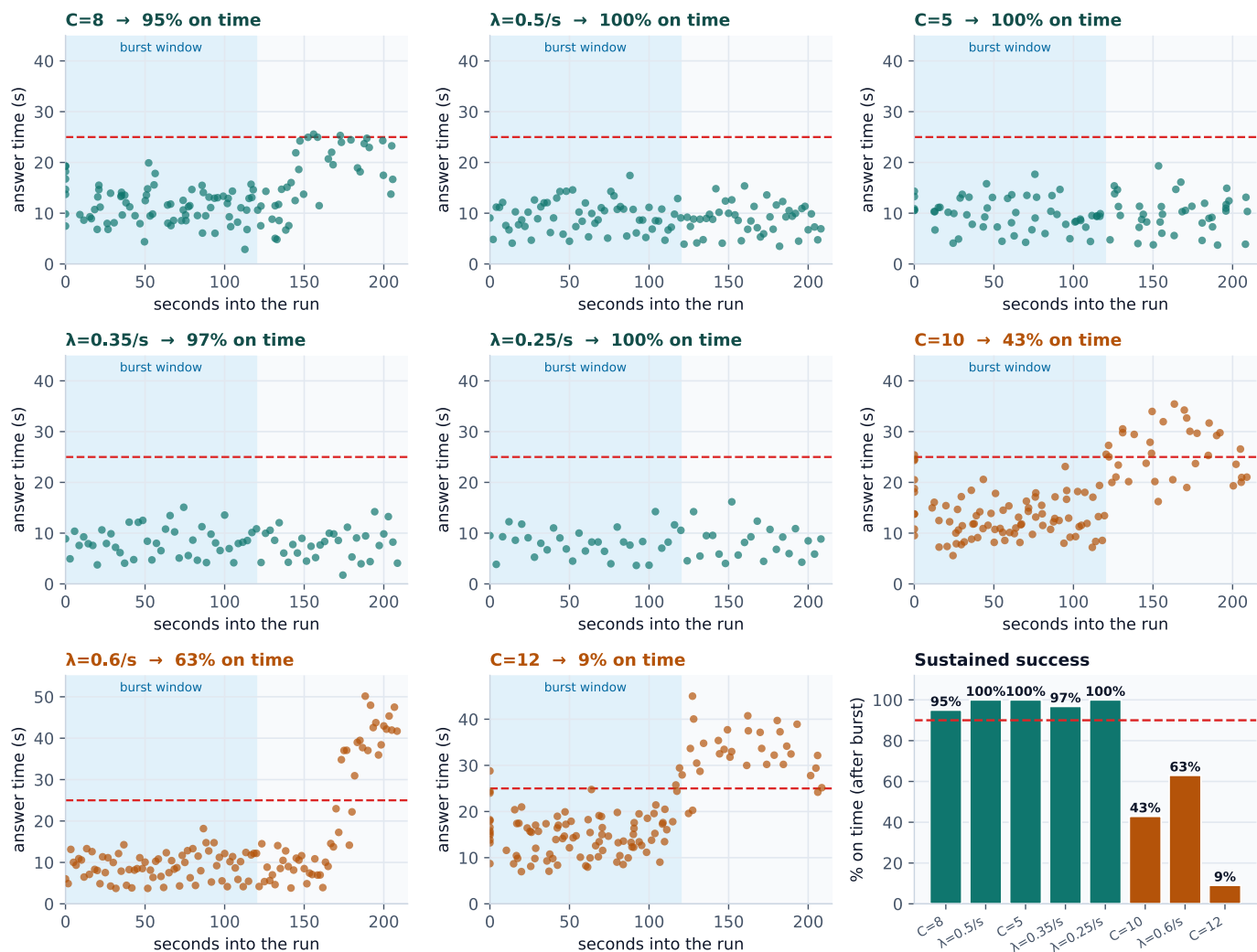


Figure 13 — 8 sustained tests. Each dot is one answer (height = seconds). The blue-tinted strip is the ~2-minute burst window; the red dashed line is the 25-second promise. Teal panels stay under the line once the burst is over; amber panels drift above it.

LOAD	ON TIME, WHOLE RUN	ON TIME, AFTER BURST	TYPICAL WAIT, AFTER BURST	SLOWEST 10% FROM	VERDICT
C=8	98%	95%	17.1 s	24.8 s	✓ sustainable
$\lambda=0.5/s$	100%	100%	8.9 s	12.4 s	✓ sustainable
C=5	100%	100%	10.3 s	14.7 s	✓ sustainable
$\lambda=0.35/s$	99%	97%	8.2 s	11.2 s	✓ sustainable
$\lambda=0.25/s$	100%	100%	8.5 s	12.1 s	✓ sustainable
C=10	83%	43%	25.5 s	32.4 s	✗ not sustainable
$\lambda=0.6/s$	84%	63%	13.9 s	42.8 s	✗ not sustainable
C=12	74%	9%	32.5 s	39.6 s	✗ not sustainable

Source for the burst rule: Google Cloud Compute Engine documentation, “E2 shared-core machine types” (cloud.google.com/compute/docs/general-purpose-machines#e2_shared-core).

9. What the server was doing meanwhile

Verdict. The VM was the bottleneck: its CPU sat at 90% or more for 32% of the measured time (peak 100%), and the “look up” step running on the VM grew from 1.2 s to 20.0 s per answer as the crowd grew, while the external LLM steps barely moved (7.3 s → 7.9 s).

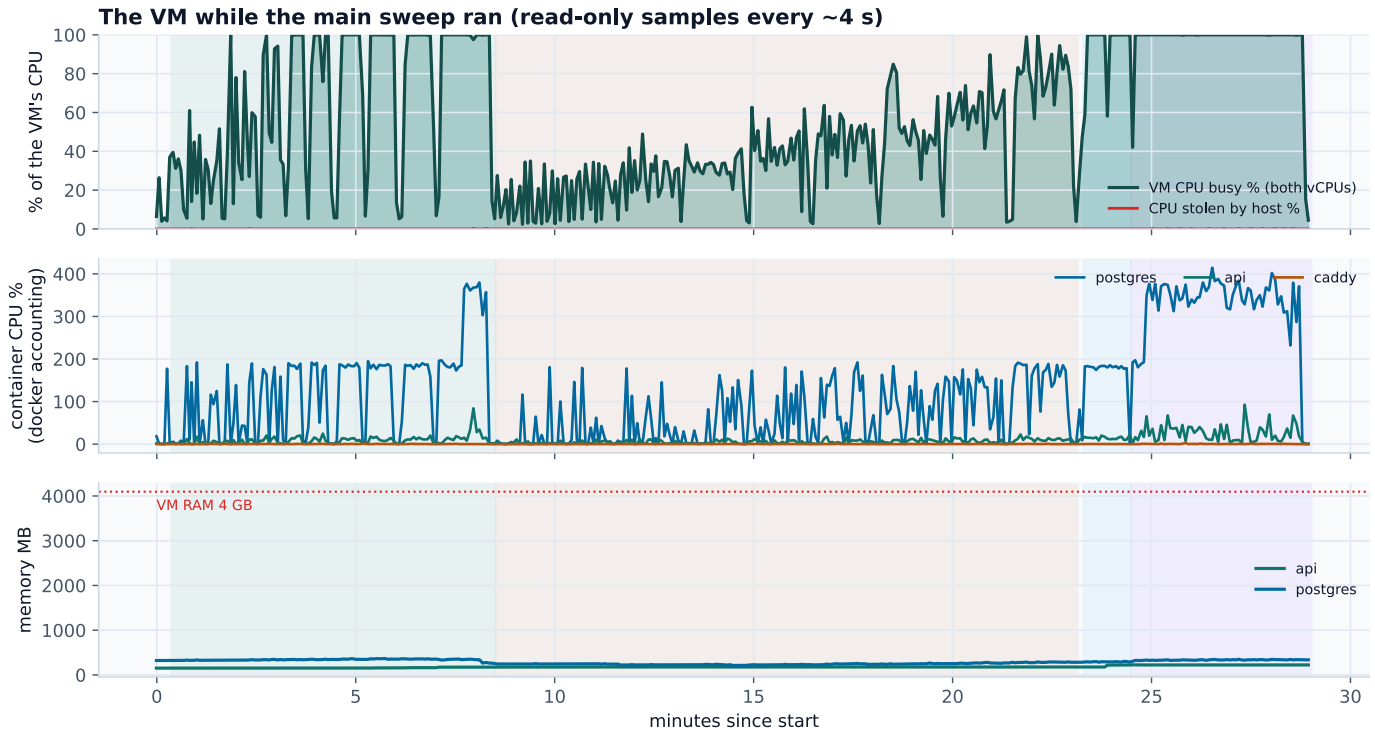


Figure 14 — VM vitals during the main sweep. Top: overall CPU use and CPU “stolen” by other tenants on the shared physical core. Middle: which container used the CPU. Bottom: memory against the 4 GB limit. Shaded bands match the test phases in Figure 4.

Peaks during the sweep — VM CPU 100% (95th percentile 100%), steal 0%, API container 93% / 226 MB, Postgres 415% / 366 MB; 411 samples. Container percentages come from Docker and are relative to *one* vCPU, so on this two-vCPU machine a single container can read above 100% (and, on a shared core, occasionally above 200%). Treat the top panel (whole-VM CPU) as the reliable saturation signal.

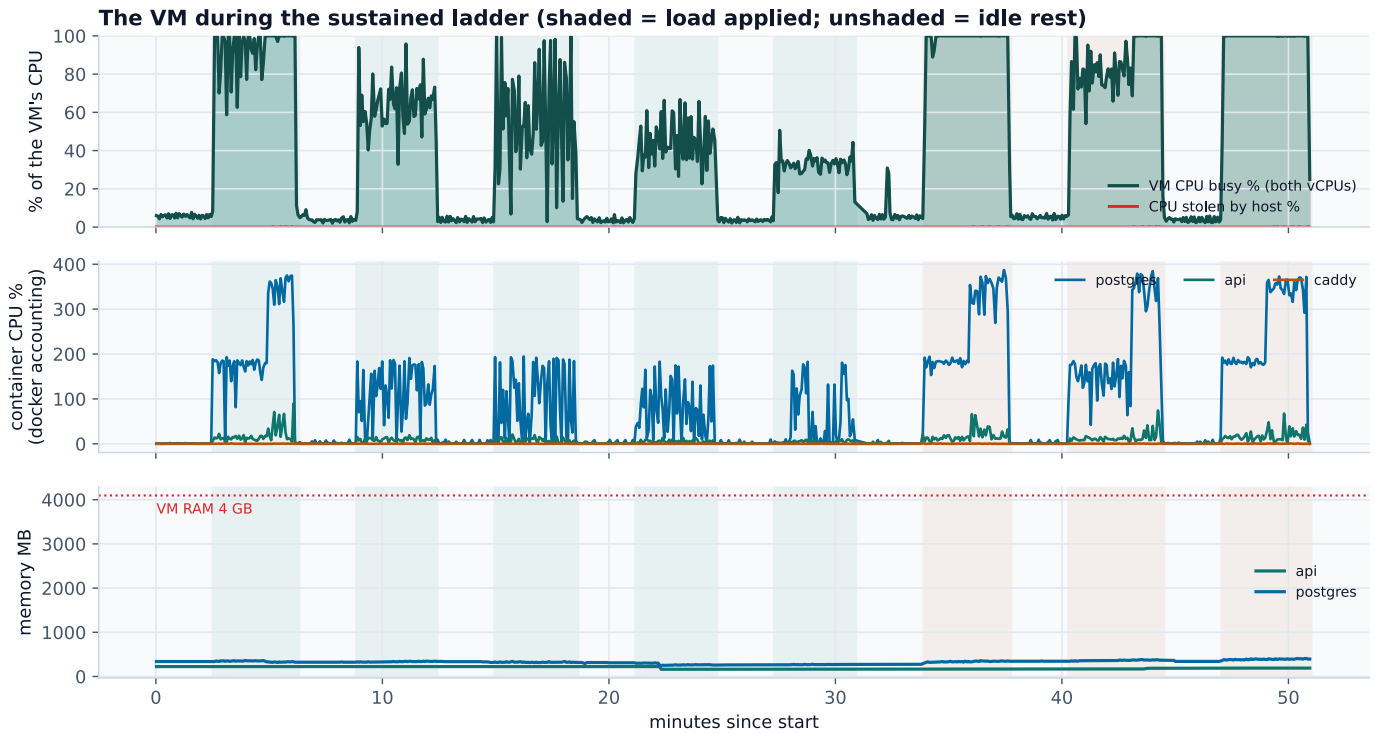


Figure 15 — VM vitals during the sustained ladder. CPU is near-idle during the rests, jumps when a level starts, and shows what the VM can keep up once the burst allowance is gone.

10. What you can and cannot say

Safe to say

- “On the shared demo (e2-medium), for a short burst up to 12 people asking fresh questions at once got 9-in-10 answers within 25 s.”
- “Sustained for several minutes, it held 8 people asking non-stop and about 30 fresh questions a minute.”
- “Repeated FAQ questions are served from the cache almost regardless of the crowd.”

Do not say

- “EdgeQuake supports N users” without naming the server, LLM, workspace, duration and date.
- That the *burst* number is the everyday capacity — on this shared-core VM it is not.
- That answer *quality* was tested — it was not.
- That production hardware would behave the same.

What would raise the ceiling

- **Give the server more CPU.** Move from the shared-core e2-medium to a dedicated-core type (for example e2-standard-4 or a compute-optimised type). Dedicated cores have no burst allowance to run out of.
- **Separate the database.** Postgres (which holds the vectors and the knowledge graph) and the API compete for the same two vCPUs; give Postgres its own machine or a managed service.
- **Make the lookup cheaper.** The “look up” step is the one that grows with the crowd — examine its queries (graph hops, number of vectors compared) before buying hardware.
- Re-run this exact protocol afterwards and compare the sustained ceiling (Figure 13) with today’s.
- Keep the answer cache on and warm it for common questions (FAQ answers cost almost nothing).

11. Reproduce it, and glossary

Everything is in the repository under `specs/153-workload-benchmark/`. Protocol: `07-highload-protocol.md`. This run's raw evidence: `measurements/2026-09-29-highload-push/` — `provider/units.jsonl` and `units-sustain.jsonl` (every request), `machines.json` and `gcp-server.json` (hardware), `telemetry.jsonl` (VM samples), `charts/` (300-DPI PNG + vector SVG).

```
make bench153-provider-highload # BENCH153_RUN_DIR=... BENCH153_API_KEY_FILE=...  
BENCH153_PROFILE=sustain make bench153-provider-highload # sustained ladder (H5)  
make bench153-highload-report
```

Glossary

Burst vs sustained

Burst: what a shared-core VM can do for its first ~2 minutes using saved-up CPU credit. Sustained: what it can do indefinitely.

Knee

The last load level that still meets the promise; just past it, waiting time climbs fast.

On time / attainment

The share of answers that arrived successfully within 25 seconds.

Typical wait (median, p50)

Half of answers were faster than this, half slower.

Slowest 10% (p90)

Nine in ten answers were faster than this number.

Closed loop

A fixed crowd; each person asks again after receiving an answer.

Open loop

Questions arrive at a steady rate regardless of whether earlier ones are done.

Cache hit

The server reused a stored answer instead of asking the LLM again.

Goodput

Good answers delivered per second — throughput that actually counts.

CPU steal

Time the VM wanted to run but the shared physical core was busy with another tenant.

Token

A word-piece; LLM work and cost are measured in tokens.

Generated by bench153 · charts by matplotlib (vector SVG embedded) · profile: push · total run 29 min.